Cryptography HW1

By: 蔡定宇 (112006265)

1. Method used

- I implemented Dynamic Programming (DP) word break with backpointers.
- The input is an uppercase A to Z string with no spaces or punctuation.
- I keep a dictionary of valid words in uppercase. Single letter words allowed are A and I only.
- Cost function: base cost is log(len(word) + 2). Function words like THE, AT, ME receive a small bonus so they are preferred.
- Tie break rule: if two paths have the same cost within a tiny epsilon, prefer the path with more segments. This encodes the preference ME AT over MEAT.
- Determinism: with the same dictionary and input, the output is the same every time.

2. Design rationale

- Transparency: in a crypto course I prefer an approach that is easy to audit and grade. DP with a tiny scoring rule is simple and deterministic.
- Requirement fit: the ME AT over MEAT preference is handled by the function word bonus and the more segments tie break.
- Portability: pure Python standard library. If words.txt exists it is used. Otherwise a small built in word list is used so the program still runs.
- Output format: all caps words separated by single spaces. Otherwise print Nonsense.

3. Algorithm workflow

- Step 1. Input check. Read one line. Must be A to Z only. Otherwise print Nonsense.
- Step 2. Dictionary. If words.txt is present, load it to uppercase and accept only A and I as single letter words. Else use a compact built in list.
- Step 3. DP forward pass.
    - Let dp[i] be the best cost to reach position i, plus a backpointer and the chosen word.
    - Let max_len be the maximum word length in the dictionary. This is pruning that keeps the inner loop bounded by O(n * L).
    - For each i, try substrings s[i:j] that are in the dictionary. Update dp[j] if the new cost is lower. On near ties prefer the split with more segments.
- Step 4. Backtrack from n to 0 using the backpointers. If there is no path, print Nonsense.
- Step 5. Print the words in uppercase joined by a single space.

4. Efficiency considerations

- Time: about O(n * L) set membership checks, where n is input length and L is the maximum dictionary word length.
- Space: O(n) for DP arrays plus the dictionary set.

Cryptography HW1

By: 蔡定宇 (112006265)

- The pruning by max_len prevents very large blow ups for long inputs.

5. Testing results

- Ambiguity preference
  - Dictionary example: A, I, AT, ME, MEAT, MEET, THE, PARK.
  - Input: MEETMEATTHEPARK -> Output: MEET ME AT THE PARK.
  - Input: MEETMEAT -> Output: MEET ME AT.
  - Input: MEAT -> Output: MEAT.
- Nonsense detection
  - Input: ABCD -> Output: Nonsense.
- Single letter policy
  - With a suitable dictionary, input IATTHEPARK-> Output: I AT THE PARK.
- Determinism
  - Re running with the same input and dictionary gives the same output.

6. References and influence

- Word Break problem framing: LeetCode 139 - https://leetcode.com/problems/word-break/
- DP variants and explanations: GeeksforGeeks Word Break (DP) - https://www.geeksforgeeks.org/dsa/word-break-problem-dp-32/
- Norvig, Natural Language Corpus Data (cost based segmentation inspiration) - https://norvig.com/ngrams/
- Grant Jenks, python wordsegment - https://github.com/grantjenks/python-wordsegment

7. Link to cryptography course topics

- This can be used after recovering a plaintext stream from a simple cipher in Chapter 2 or 3. Once a candidate key yields a raw uppercase sequence, segmentation helps judge whether the text is plausible.

8. Author notes

- I did not use an external word list for grading by default. The built in list is enough for the sample and basic checks. If a larger list is needed I can add words.txt and cite its source.
- I prefer simplicity for grading. For this course I avoided machine learning. If I had more time I might add a very small bigram bonus table, but I would keep the method deterministic.

Cryptography HW1

By: 蔡定宇 (112006265)

9. Why determinism

- Fair grading. The same input and dictionary produces the same output on every run.
- Easy to audit. TAs can step through the DP table and see why the chosen path wins.
- Consistent with crypto style methods where each step is defined and reproducible.